

ТЗ: модуль «AI-тренер разговорной речи»

Стек: React 18 + Vite + Zustand (frontend), Fastify 4 + PostgreSQL 16 raw SQL (backend), Anthropic/OpenAI SDK уже подключены. Основа — аналитика в [AI-тренер-разговорной-речи.md](#).

0. Цель

Дать ученику потренировать устную/письменную речь на лексике конкретного урока с AI-персонажем, с разбором ошибок и отчётом. Встраивается в существующий цикл урока, а не живёт отдельным продуктом. **Тренер обязан помнить всю историю общения с учеником** — это сквозное требование, а не фича поздних спринтов (см. §3).

1. Пользовательские сценарии по спринтам

Спринт 0 — текстовый тренер

1. На экране урока (там же, где 7 плиток тренировок — Выбери ответ, Флеш-карта и т.д.) появляется 8-я плитка «Поговори с тренером».
2. Ученик выбирает персонажа (3-5 карточек: пол/имя/эмоции или фото-заглушка) и тему: «Слова урока N» (дефолт, предзаполнено словами текущего урока), «Свободный диалог», «Сценарий» (кафе/магазин/знакомство — список сценариев, привязанных к lesson_level).
3. Открывается чат-экран: тренер пишет первую реплику на немецком **с учётом предыдущих разговоров** (см. §3) — например, спрашивает про то, что обсуждали в прошлый раз, или сразу отмечает прогресс по повторяющейся ошибке.
4. Ученик отвечает текстом. После каждой реплики — инлайн-исправление (зачёркнутый неверный вариант + правильный) и краткое объяснение на русском, если есть ошибка.
5. Диалог заканчивается по кнопке «Завершить» или через N реплик → экран отчёта: список использованных слов урока, список ошибок с объяснениями, кнопка «Пройти ещё раз» / «Вернуться к уроку». При завершении сессии память тренера обновляется (§3).

Спринт 1 — голос 6. Кнопка микрофона: ученик надиктовывает ответ (Web Speech API, как в «Произношение»/«Диктант»), тренер зачитывает свои реплики через TTS. 7. Настройка темпа речи тренера (обычный/медленный) — хранится в user_settings. 8. Bilingual mode: тумблер «сначала на русском» — тренер перед немецкой фразой на старте диалога проговаривает её смысл по-русски.

Спринт 2 — разбор произношения + встраивание в SRS 9. Если в модуле «Произношение» уже есть speech-assessment — переиспользовать тот же вызов для оценки реплик в диалоге, добавить фонемную подсветку в отчёт. 10. На странице Словаря у слов со статусом «Изучается» — кнопка «закрепить в разговоре»: открывает тренера с принудительным фокусом на выбранных словах. 11. Раздел «Разговорник» получает отдельный вход в тренера без ограничения лексикой урока.

Спринт 3 — видео-аватар (премиум) 12. Персонажи получают реальные фото (по национальностям), при разговоре голосом лицо анимируется (D-ID/HeyGen LiveAvatar). 13. Лимит минут аватара в месяц на пользователя, после лимита — откат на голос без видео. 14. В разделе «Учитель → Отчёт» — сводка по разговорным сессиям учеников.

2. Модель данных (новые таблицы, миграции в `backend/src/db/migrations`)

-- персонажи тренера (сид-данные, редактируются админом)

```
ai_trainer_personas (  
  
    id, name, gender, nationality, avatar_url, photo_url_for_video,  
  
    voice_id, speech_rate_default, is_premium, created_at  
  
)
```

-- сессии диалога

```
ai_trainer_sessions (  
  
    id, user_id, persona_id, lesson_id nullable, mode enum('lesson_vocab','scenario','free_talk'),  
  
    scenario_id nullable, target_word_ids int[] nullable,  
  
    status enum('active','finished'), started_at, finished_at  
  
)
```

-- реплики (полная сырая история — источник истины)

```
ai_trainer_messages (  
  
    id, session_id, role enum('trainer','user'), text,
```

```

corrected_text nullable, correction_explanation nullable,

pronunciation_score nullable, audio_url nullable, created_at

)

-- сценарии (кафе/магазин/знакомство)

ai_trainer_scenarios (

    id, title, description, min_lesson_level, system_prompt, created_at

)

-- отчёт по сессии

ai_trainer_reports (

    id, session_id, words_practiced int[], words_correct int[],

    mistakes_summary jsonb, overall_score, created_at

)

-- ПАМЯТЬ О УЧЕНИКЕ (сквозная, между всеми сессиями) — см. §3

ai_trainer_memory (

    id, user_id unique,

    summary_text,          -- накопительная выжимка "кто этот ученик, что обсуждали"

    known_facts jsonb,     -- {"имя": "...", "город": "...", "интересы": [...]}

    recurring_mistakes jsonb, -- [{type, example, times_seen, last_seen_at}]

    topics_covered jsonb,  -- [{topic, session_id, date}]

    sessions_total int,

    last_session_id,

```

updated_at

)

`user_settings` — добавить поля `ai_trainer_speech_rate`,
`ai_trainer_bilingual_mode`.

3. Память между сессиями (обязательное требование)

Проблема. Просто хранить сообщения в `ai_trainer_messages` недостаточно — это архив, а не память. Чтобы тренер реально «помнил», при старте *каждой новой* сессии в промпт нужно класть не весь сырой лог всех прошлых разговоров (он будет расти неограниченно и упрётся в лимит контекста и в стоимость токенов), а компактную, постоянно обновляемую выжимку.

Решение — накопительная суммаризация:

1. При завершении сессии (`POST /sessions/:id/finish`) сервер, помимо генерации отчёта, вызывает модель ещё раз: на вход — текущий `ai_trainer_memory.summary_text` (если это не первая сессия) + полный лог только что законченной сессии из `ai_trainer_messages`. На выходе — обновлённые `summary_text`, `known_facts`, `recurring_mistakes` (мёржаются: новая ошибка добавляется, повторная — увеличивает `times_seen`), `topics_covered`. Результат перезаписывает строку в `ai_trainer_memory` (upsert по `user_id`).
2. Это отдельный, самостоятельный вызов AI (не блокирует показ отчёта ученику — можно выполнить асинхронно сразу после ответа на `/finish`, или синхронно, если задержка в 1-2 сек приемлема).
3. При старте новой сессии (`POST /sessions`) система подтягивает `ai_trainer_memory` по `user_id` и кладёт в system-промпт: `summary_text`, `known_facts`, топ-3 `recurring_mistakes` по частоте. Персонаж явно инструктируется использовать эту информацию (упомянуть, что помнит ученика, ссылаться на прошлые темы, целенаправленно давать практику на повторяющихся ошибках).
4. Память — на уровне ученика (`user_id`), а не персонажа: если ученик сменил тренера-аватара, новый персонаж всё равно "знает" историю — так делает Duolingo/Praktika, и это ожидаемо пользователем. Если продукту важнее, чтобы каждый персонаж помнил отдельно — заменить `unique(user_id)` на `unique(user_id, persona_id)`, но по умолчанию рекомендуется общая память.
5. Сырая история (`ai_trainer_messages`) остаётся доступна отдельно — на неё опирается экран «История сессий» (`GET /api/ai-trainer/sessions`), где ученик может открыть и перечитать любой прошлый разговор целиком. Память (`ai_trainer_memory`) — это рабочий контекст для AI, а не то, что показывается

ученику напрямую (хотя `known_facts/recurring_mistakes` можно вывести в профиле как «тренер запомнил» — опционально).

6. Если объём `recurring_mistakes/topics_covered` начнёт расти неограниченно за месяцы использования — предусмотреть в логике суммаризации схлопывание старых/неактуальных записей (например, ошибки, не повторявшиеся 30+ дней, понижаются в приоритете или убираются из топ-N, который уходит в промпт).

4. API (`backend/src/routes/aiTrainer.js`)

Метод	Путь	Назначение
GET	<code>/api/ai-trainer/personas</code>	список персонажей (фильтр premium по подписке)
GET	<code>/api/ai-trainer/scenarios</code>	список сценариев
POST	<code>/api/ai-trainer/sessions</code>	создать сессию; сервер подтягивает <code>ai_trainer_memory</code> для system-промпта
POST	<code>/api/ai-trainer/sessions/:id/messages</code>	отправить реплику ученика → ответ тренера + коррекция (SSE-стрим, как в <code>chat</code>)
POST	<code>/api/ai-trainer/sessions/:id/finish</code>	завершить сессию, сгенерировать отчёт + обновить <code>ai_trainer_memory</code>
GET	<code>/api/ai-trainer/sessions/:id/report</code>	получить отчёт
GET	<code>/api/ai-trainer/sessions</code>	история сессий ученика (полный лог по каждой)
GET	<code>/api/ai-trainer/memory</code>	текущая память тренера о пользователе (для UI «тренер запомнил» и для отладки)

Метод	Путь	Назначение
POST	<code>/api/ai-trainer/sessions/:id/tts</code>	(Спринт1) озвучить реплику тренера
POST	<code>/api/ai-trainer/sessions/:id/avatar-stream</code>	(Спринт3) прокси к D-ID/HeyGen

Все ключи AI/TTS/аватар-провайдеров — только в backend `.env`, вызовы идут через сервер.

5. AI-логика

- System-промпт собирается из: персонажа, режима (`lesson_vocab` → список слов урока обязателен), уровня ученика, **памяти о ученике из `ai_trainer_memory`**.
- Ответ модели в диалоге — структурированный (tool use / JSON): `{reply_de, correction: {original, corrected, explanation_ru} | null, words_used: []}`.
- Обновление памяти (§3) — отдельный структурированный вызов с своей JSON-схемой ответа (`summary_text, known_facts, recurring_mistakes[]`, `topics_covered[]`).
- Anthropic SDK уже используется в проекте — переиспользовать тот же клиент/обёртку.

6. Frontend

- `src/pages/AiTrainer/` — `PersonaPicker.jsx`, `TopicPicker.jsx`, `ChatSession.jsx`, `ReportView.jsx`, `SessionHistory.jsx`
- Zustand-стор `aiTrainerStore.js`: текущая сессия, список сообщений, статус загрузки, настройки голоса
- Плитка на экране урока — по образцу существующих 7 плиток
- Голосовой ввод — обёртка над Web Speech API, как в «Диктант»/«Произношение», вынести в общий хук `useSpeechRecognition`, если его ещё нет
- Опционально: в шапке чата небольшая плашка «Тренер помнит: были ошибки с артиклями, обсуждали переезд в Берлин» — из `known_facts/recurring_mistakes`, повышает ощущение непрерывности для ученика

7. Не входит в MVP (Спринт 0-1)

Видео-аватар, экзаменационные рубрики (Goethe/TestDaF), учительский отчёт. Память между сессиями (§3) — **входит в MVP с самого начала**, не откладывается.

8. Тестирование

- Vitest: генерация отчёта из массива messages — чистая функция, юнит-тесты
- Vitest: логика мёржа памяти (новая ошибка добавляется, повторная — инкрементит `times_seen`, старое схлопывается) — отдельно от вызова AI, чистая функция над `known_facts/recurring_mistakes`
- Testing Library: флоу выбора персонажа → диалог → коррекция → завершение → **новая сессия ссылается на данные из предыдущей** (интеграционная проверка, что память реально применяется)
- Ручная проверка: system-промпт ограничивает тренера словами урока и не «забывает» память при переключении режима (`lesson_vocab` / `free_talk` / `scenario`)

9. Метрики успеха

Доля учеников, попробовавших тренера хотя бы раз после урока; среднее число реплик за сессию; % слов урока, использованных верно, до/после повторных попыток; доля сессий, где тренер корректно сослался на память (ручная выборочная проверка на первых неделях).